

# Week 3 - Extract, transform, and load (ETL)

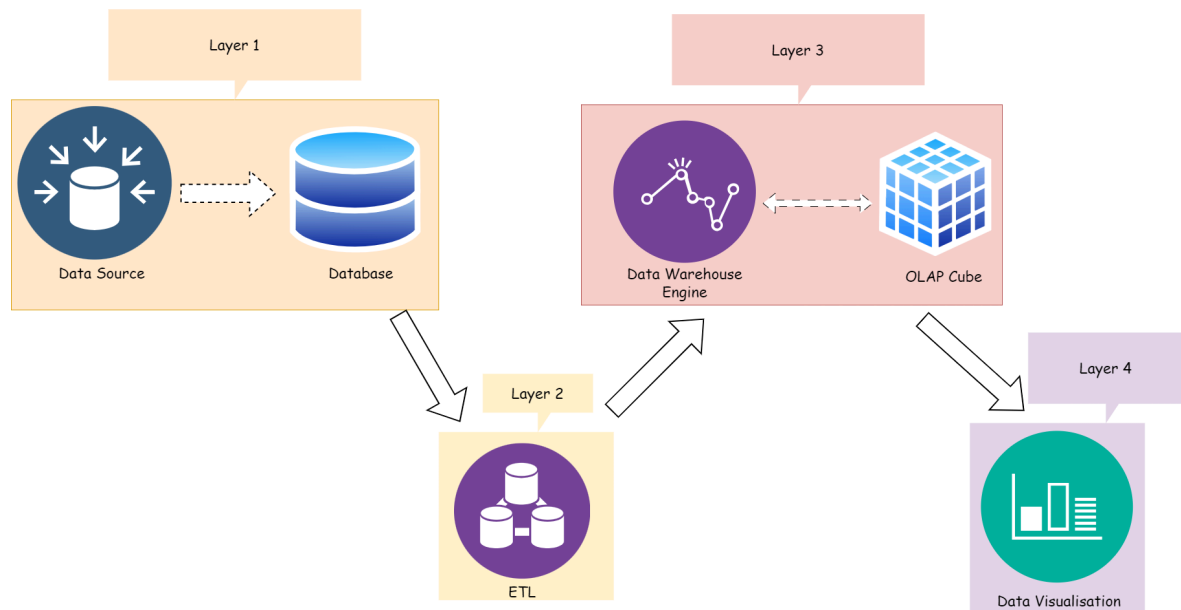
[Copy](#)

This week, we will practice the the ETL process, particularly data extraction and transformation (**Layer 2 in the OLAP Architecture below**).

This week, the lab sheet contains 2 parts.

- Part 1: Self-reading. The aim of Part 1 is to help you understand the basic concepts of ETL and data quality.
- Part 2: Practice ETL skills on a simple dataset.

## OLAP Architecture



OLAP Architecture

# Part 1: Theoretical Concepts

## What is ETL?

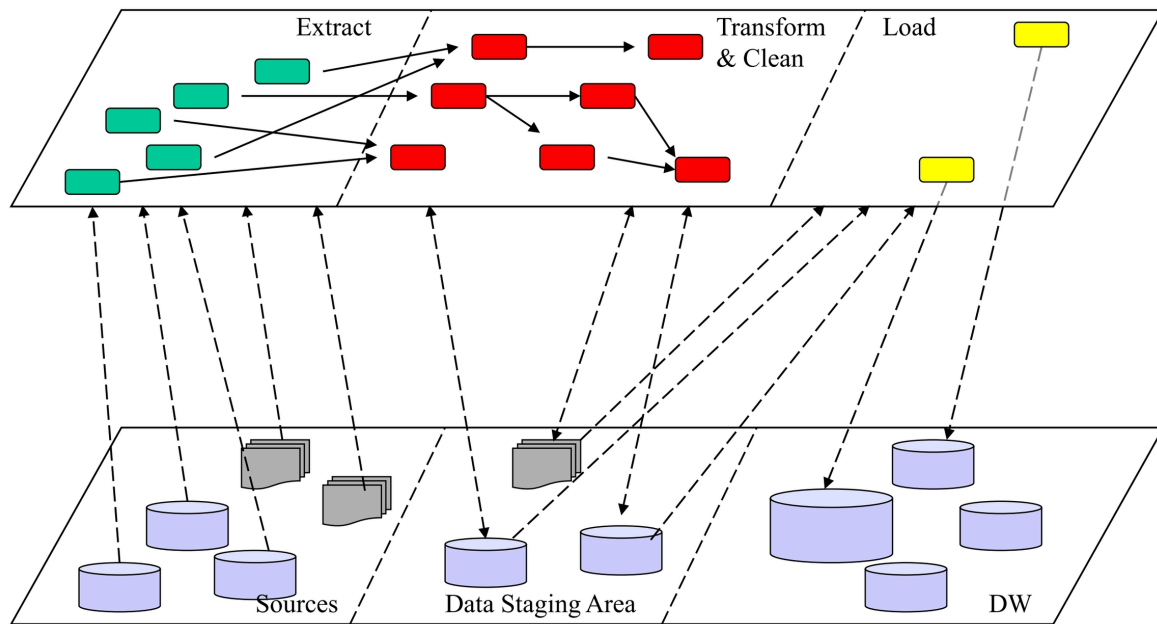
- **Extract**
  - The process of fetching relevant data from the source systems.
- **Transform**
  - The process in which we apply transformation such as
    - aggregation, joining with other datasets, applying rules, splitting or merging results, lookup, join, pivot, and
    - applying the data reduction techniques, and many others.
- **Load**
  - The process of loading data into data warehouse destination tables such as fact and dimensions.
  - Building additional structures to improve system performance.

The source data should be loaded into the data warehouse in a **consistent** way and be **reliable**.

## ETL Overview

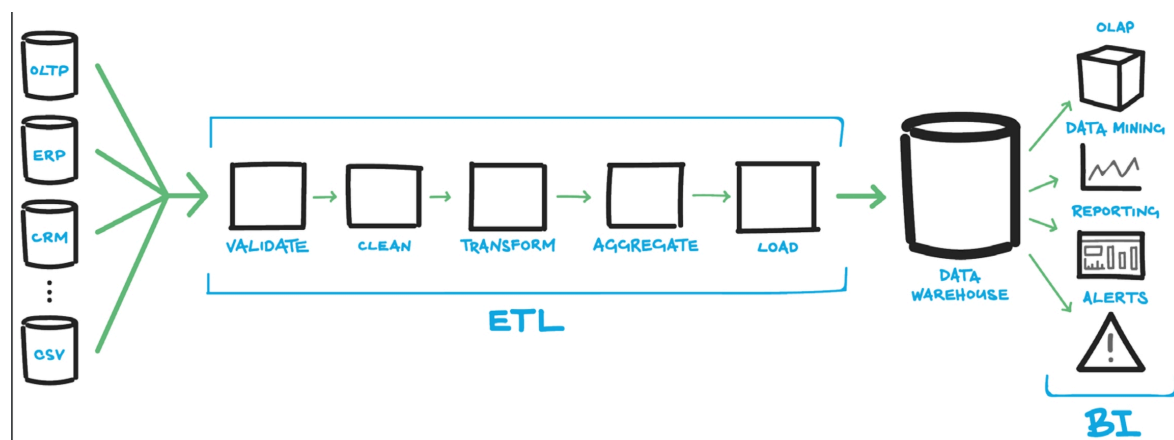
Architecture of data warehousing:

Data sources → Data staging area → Data warehouse



ETL Overview

## ETL Lifecycle



ETL Lifecycle

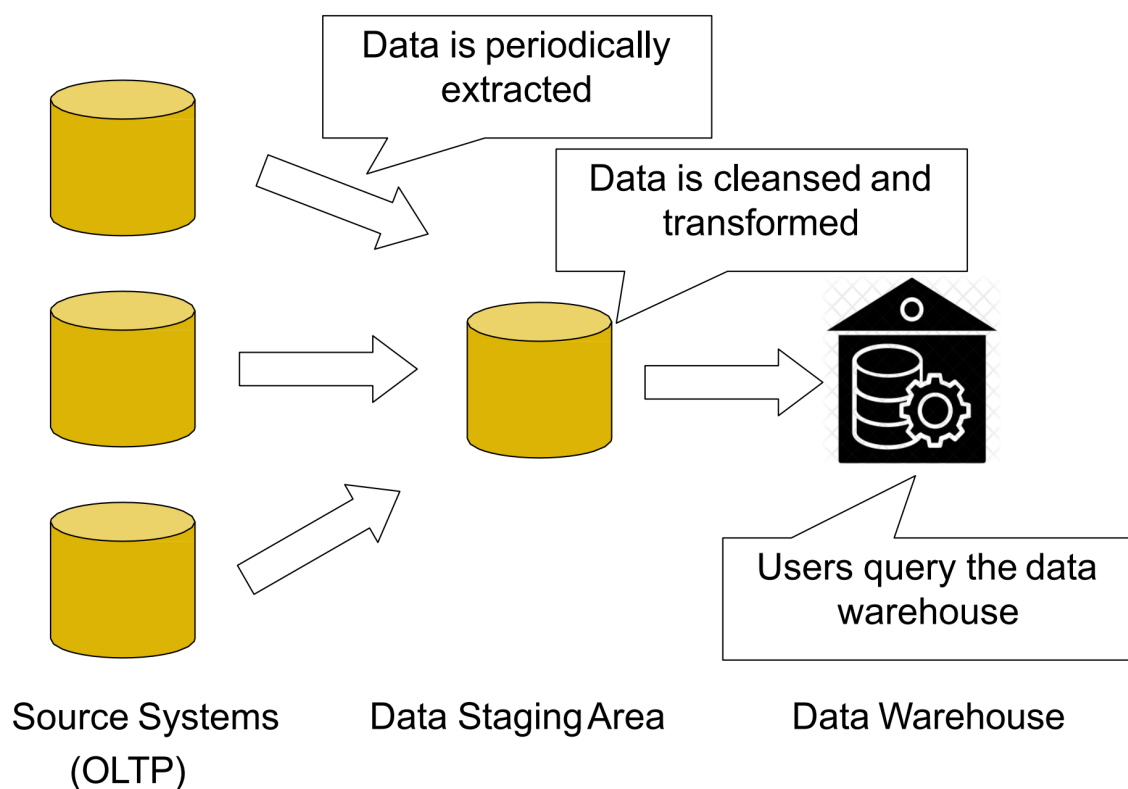
## Extract and Transform in ETL

Data transfer or ETL is an important component of the BI and data warehousing system.

- The extract step should be able to pick data from all these kinds of data sources.

- Data extraction is a process of fetching relevant data from the source systems. Data extraction is a process of fetching relevant data from the source systems. Data might come from different sources such as an Excel file, a .csv file, a relational database, or a website.
- The transformation step should be careful to ensure this data is consistent.
  - Build keys; apply data-cleansing rules on the dataset.
- Fetch the data in an incremental method.
  - Data in the source system might be removed or replaced by newer data records.
  - Some data rows may appear only once in the data source and be replaced with new data.

## Loading Data to the Data Warehouse



Loading data to the data warehouse

## ETL Major Steps

- Determine the data you need and don't need.
- Determine all the data sources, both internal and external.
- Establish comprehensive data extraction, cleansing rules and transformation rules.
- Plan for aggregated tables.
- Organise data staging area and test tools.
- Write a procedure for all data loads.
- Loading dimension tables and fact tables.

## Data Staging

A staging area ensures that

- We don't spend any extra activities in the extract step to convert or transform data, and also.
- The data won't be loaded all at once into the memory.

Extracting data from sources into an integrated database comes with some challenges.

- For very big datasets, fetching data from sources shouldn't be combined with any kind of conversion or transformation.
  - The main reason is that it will reduce the performance and slow down the whole ETL process.
  - Loading a very large dataset into memory will require a high amount of server resources that are not available at all times

## Logical Data Map

The logical data map describes the relationship between the extreme starting points and the extreme ending points of the ETL system usually presented in a table or spreadsheet.

### Components of Logical Data Map

The logical data map is usually presented in a table or spreadsheet format and includes the following specific components:

- **Target table name:** The physical name of the table as it appears in a data warehouse.
- **Target column name:** The name of the column in a data warehouse table.
- **Table type:** Indicates if the table is a fact or dimension.

### The Logical Data Map Example

| Mapping<br>Change<br>Date | TARGET               |                          |                             |                              |                                      | SOURCE               |                                                     |                                             |                                                                   |                                                        |                                                                               |
|---------------------------|----------------------|--------------------------|-----------------------------|------------------------------|--------------------------------------|----------------------|-----------------------------------------------------|---------------------------------------------|-------------------------------------------------------------------|--------------------------------------------------------|-------------------------------------------------------------------------------|
|                           | Target Table         | Target Column            | Nullable                    | PK                           | Data-type, Length                    | Source Table         | Source Column                                       | Data-type, Length                           | Expression, Transformation                                        | Default Value                                          | Error Types and Handling                                                      |
| Y/M/D                     | Name of target table | Target table column name | Whether a field can be null | Primary key field for target | Data type & length for target column | Name of source table | Column in source table from which data is extracted | Data type and length for this source column | Decodes, aggregates, conversions, if statements, lookup functions | Value to use in target field when source field is null | Used to document Not null, value if looked up, pk, fk, etc...comments, issues |

The logical data map example

### Hints of Logical Data Map

- **Have a plan:** the foundation of the metadata
- **Identify data source candidates:** Identify the likely candidate data sources you believe will support the decisions needed by the business community

- **Analyse source systems with a data-profiling tool:** Detected data anomalies must be documented, and best efforts must be made to apply appropriate business rules to rectify data before it is loaded into the data warehouse.

## Data Extraction: Data Discovery Phase

**Context:** The analysis of the source system is usually divided into two major phases:

- The data discovery phase
- The anomaly detection phase

Data Discovery Phase key criterion for the success of the data warehouse is the cleanliness and cohesiveness of the data within it.

Once you understand what the target needs to look like, you need to identify and examine the data sources.

## Physical Data Extraction

### Online Extraction

- Data is extracted directly from the source system.
- May access source tables through an intermediate system.
- The intermediate system is usually similar to the source system.

### Offline Extraction

- Data NOT extracted directly from the source system, instead staged explicitly outside the original source system.
- Data is either already structured or was created by an extraction routine.

## Logical Data Extraction

### Full Extraction

- The data is extracted completely from the source system.
- No need to keep track of changes.
- Source data is made available as-is with any additional information.

### Incremental Extraction

- Data is extracted after a well-defined point/event in time.
- The mechanism used to reflect/record the temporal changes in data (column or table).
- Sometimes entire tables off-loaded from a source system into the Data Warehouse.
- Can have significant performance impacts on the data warehouse server.

## Data Transformation

The data after transformation should be

- Correct
- Unambiguous
- Consistent
- Complete

Data quality checks are run at 2 places

- after extraction and
- after cleaning and confirming



## Data Transformation: Cleaning Data

### Anomaly Detection

- E.g. a row significantly different from the rest

### Column Property Enforcement

- Null Values in required columns
- Numeric values that fall outside of expected highs and lows
- The length of an attribute is exceptionally short/long
- Columns with certain values outside of discrete valid value sets
- Adherence to a required pattern/member of a set of pattern

## Transformation - Confirming

Confirmation can be done in various ways

- Structure Enforcement
- Tables have proper primary and foreign keys
- Obey referential integrity
- Data and rule value enforcement
- Simple business rules
- Logical data checks

## What is data quality?

Data quality is a perception or an assessment of data's fitness to serve its purpose in a given context.



Data quality lifecycle

Data quality is described by several dimensions

- Correctness/Accuracy
- Consistency
- Completeness
- Timeliness
- Integrity/Validity
  - Integrity ensures that all data in a database can be traced and connected to other data

## Data Cleansing and Matching

It is the process of identifying and correcting dirty data. Dirty data means incomplete, wrong, duplicate, or out-of-date data. The data warehousing community often uses the words cleanse or scrub rather than clean

Data matching is to determine that one data item is the same as another data item. Data matching is used to identify duplicate records. Matching is particularly relevant for character-based data types.

- Numeric Data Types
- Datetime Data Types
  - For example, is 03/01/2021 the same as 01/03/2021? Is it the same as 2021-03-01T00:00:00Z+06?
  - We need some logic here to match them, such as by comparing the components or by comparing the date, month, and year.
- Character-based Data Types
  - For example, a customer named "Mr. Aleck Stevenson". We need to match/recognise that "Mr. Aleck Stevenson" is the same as "Mr. S Aleck", "Mr. Stevenson Aleck" and "Mr. Alec Stevenson".

## Data Quality Validation

Data quality rules (data validation) are filters to prevent dirty data from getting into the warehouse. Based on the data location, there are three kinds of validation

- Incoming data validation
  - Checking the incoming data only on its own, without referring to the data already in the warehouse.
  - These rules verify that the data from the source systems is valid, e.g. within the expected range and in the right format
- Cross-Reference Validation
  - Cross-reference validation is where we check the incoming data against the data in the warehouse.
  - To make sure that the value of the incoming data is within a certain range that is calculated based on data in the warehouse
  - Like incoming data validation, cross-reference validation is performed on the fly when the data is being loaded into the warehouse.
- Data Warehouse Internal Validation
  - Data warehouse internal validation is where we check the data already in the warehouse.
    - We don't check the incoming data

- Unlike the previous two, data warehouse internal validation is performed after the incoming data is fully loaded into the warehouse.
- The purpose of doing this is to verify the quality of the data in the warehouse at the aggregate level.
  - This can be done by comparing the totals over a period of time against a known standard value.

## Part 2: ETL Practices

Consider the workflow and try to practice the solution for the scenario below.

### Description

The dataset below is an original transaction dataset. The file contains 20 transactions with the following columns:

- Transaction\_ID: Unique identifier for each transaction
- Date: Date of purchase
- Customer\_Name: Name of the customer
- Gender: Gender of the customer (M/F)
- Customer\_Type: Type of customer (Regular/Premium)
- Product: Name of the product purchased
- Product\_Category: Category of the product
- Unit\_Price: Price per unit of the product
- Quantity: Number of units purchased
- Discount: Discount amount applied to the transaction
- City: The city where the purchase was made
- State: State/province where the purchase was made
- Country: The country where the purchase was made

 2KB

Transaction Dataset.csv

 Download

 Open

Transaction Dataset

The dimension tables and fact table below were processed from the transaction dataset above.

 163B

Dim\_Customer.csv

 Download

 Open

Customer Dimension Table

 103B

Dim\_Date.csv

 Download

 Open

Date Dimension Table

 109B

Dim\_Location.csv

 Download

 Open

Location Dimension Table

 132B

Dim\_Product.csv

 Download

 Open

Product Dimension Table

 766B

Fact\_Sales.csv

 Download

 Open

Sales Fact Table

## Task

**Can you reproduce these dimension tables and the fact table from the original transaction dataset?**

## Task Demonstration

A demonstration video is available in the [subpage](#).

! You can use any programming language, such as Python or R, or Excel to complete the above task.



Data Warehousing  
Lab sheets



## Part 3: Databricks ETL Pipeline

### Overview

In this section, we implement a simple ETL pipeline using **Databricks Free Edition** and **PySpark**. We ingest the **raw transaction dataset** from Part 2 ( `Transaction Dataset.csv` ) and transform it into a small **star schema** consisting of four Dimension tables and one Fact table:

- `Dim_Customer(CustomerID, CustomerName, Gender, CustomerType)`
- `Dim_Date(DateID, Date, Month, Year)`
- `Dim_Location(LocationID, City, State, Country)`
- `Dim_Product(ProductID, ProductName, Category)`
- `Fact_Sales(SalesID, ProductID, CustomerID, DateID, LocationID, Quantity, TotalPrice, Discount, ActualPrice)`

The workflow follows the standard ETL stages **Extract** → **Transform** → **Load** → **Validate**.

## Step 1: Initialize Environment

**Note:** Databricks Free Edition now manages compute resources automatically. You do not need to create a cluster manually.

1. Click **+ New** (top-left sidebar) → **Notebook**.
2. Wait for the notebook to load.
3. Check the top-right corner:
  - Once you see a **green circle/indicator** (e.g., labeled "**Serverless**"), your environment is ready.

**Verification:** Type `print(spark.version)` in the first cell and run it. If it prints a version number (e.g., 3.x.x or 4.x.x), you are ready to proceed.



## Step 2: Upload Data

1. Navigate to **+ New** → **Add or upload data**.
2. Click on **Create or modify table** (the icon with a blue upload arrow).
3. Click **Browse** and select your local `Transaction Dataset.csv` file.
4. Once the preview appears, click **Create table** at the bottom right.
  - *Note: This action uploads the file to the default storage and registers it as a table.*
5. **Return to your Notebook** for the next step.

Add data >

Create or modify table from file upload

Transaction Dataset.csv uploaded 1.97KB

Create new table

Preview mode

Catalog

workspace

Schema

default

Table name

transaction\_dataset

Advanced attributes

Serverless Starter Warehouse

Previewing 20 rows, 13 columns

| Transaction_ID | Date       | Customer_Name  | Gender | Customer_Type | Product          | Product_Category | Unit_Price | Quantity |
|----------------|------------|----------------|--------|---------------|------------------|------------------|------------|----------|
| T1001          | 2025-01-15 | John Smith     | M      | Regular       | Wireless Earbuds | Electronics      | 79.99      | 1        |
| T1002          | 2025-01-15 | Emily Johnson  | F      | Premium       | Yoga Mat         | Fitness          | 29.99      | 2        |
| T1003          | 2025-01-20 | Marie Tremblay | F      | Regular       | Coffee Maker     | Home             | 99.99      | 1        |
| T1004          | 2025-01-20 | Michael Brown  | M      | Premium       | Smart Watch      | Electronics      | 249.99     | 1        |
| T1005          | 2025-02-05 | Emily Johnson  | F      | Premium       | Wireless Earbuds | Electronics      | 79.99      | 1        |
| T1006          | 2025-02-05 | John Smith     | M      | Regular       | Yoga Mat         | Fitness          | 29.99      | 3        |
| T1007          | 2025-02-10 | Michael Brown  | M      | Premium       | Coffee Maker     | Home             | 99.99      | 1        |
| T1008          | 2025-02-10 | Marie Tremblay | F      | Regular       | Smart Watch      | Electronics      | 249.99     | 1        |
| T1009          | 2025-01-15 | John Smith     | M      | Regular       | Coffee Maker     | Home             | 99.99      | 2        |
| T1010          | 2025-01-20 | Emily Johnson  | F      | Premium       | Wireless Earbuds | Electronics      | 79.99      | 1        |
| T1011          | 2025-02-05 | Marie Tremblay | F      | Regular       | Yoga Mat         | Fitness          | 29.99      | 4        |
| T1012          | 2025-02-10 | Michael Brown  | M      | Premium       | Wireless Earbuds | Electronics      | 79.99      | 2        |
| T1013          | 2025-01-15 | Emily Johnson  | F      | Premium       | Smart Watch      | Electronics      | 249.99     | 1        |
| T1014          | 2025-01-20 | John Smith     | M      | Regular       | Yoga Mat         | Fitness          | 29.99      | 2        |
| T1015          | 2025-02-05 | Michael Brown  | M      | Premium       | Coffee Maker     | Home             | 99.99      | 3        |
| T1016          | 2025-02-10 | Marie Tremblay | F      | Regular       | Wireless Earbuds | Electronics      | 79.99      | 1        |

Cancel

Create table

Catalog Explorer > workspace > default >

transaction\_dataset

Open in

Overview

Sample Data

Details

Permissions

Policies

History

Lineage

Insights

Quality

AI Suggested Description

The table contains transaction data related to customer purchases. It includes details such as the customer name, product information, pricing, and location. Use cases for this data include analyzing sales trends, understanding customer demographics, and evaluating product performance across different regions.

Accept

Edit

Send feedback

Filter columns...

AI generate

| Column           | Type   | Comment | Tags | Column masking |
|------------------|--------|---------|------|----------------|
| Transaction_ID   | string |         |      |                |
| Date             | date   |         |      |                |
| Customer_Name    | string |         |      |                |
| Gender           | string |         |      |                |
| Customer_Type    | string |         |      |                |
| Product          | string |         |      |                |
| Product_Category | string |         |      |                |
| Unit_Price       | double |         |      |                |
| Quantity         | bigint |         |      |                |
| Discount         | double |         |      |                |
| City             | string |         |      |                |



## Step 3: Ingest Raw Data

In this step, we verify the table creation and load the data into a Spark DataFrame for processing.

```
# 1. Define the table name
# Note: This must match the table name created in Step 2 (usually lowercase)
table_name = "transaction_dataset"

# 2. Load data directly from the registered table
df_raw = spark.read.table(table_name)

# 3. Validation
print(f"Successfully loaded table: {table_name}")
print(f"Total records: {df_raw.count()}")

# 4. Preview the data
display(df_raw.limit(5))
```

Successfully loaded table: transaction\_dataset  
Total records: 20

| Transaction_ID | Date       | Customer_Name  | Gender | Customer_Type | Product          | Product_Category | Unit_Price | Quantity | Discount |
|----------------|------------|----------------|--------|---------------|------------------|------------------|------------|----------|----------|
| T1001          | 2025-01-15 | John Smith     | M      | Regular       | Wireless Earbuds | Electronics      | 79.99      | 1        |          |
| T1002          | 2025-01-15 | Emily Johnson  | F      | Premium       | Yoga Mat         | Fitness          | 29.99      | 2        |          |
| T1003          | 2025-01-20 | Marie Tremblay | F      | Regular       | Coffee Maker     | Home             | 99.99      | 1        |          |
| T1004          | 2025-01-20 | Michael Brown  | M      | Premium       | Smart Watch      | Electronics      | 249.99     | 1        |          |

5 rows | 11.94s runtime

Refreshed now

## Step 4: Transform (Build Dimension Tables and Fact Table)

In this step, we clean the raw data (trim strings, cast types, parse dates), generate **surrogate keys** for each dimension, and then construct the fact table by joining the raw transactions to the dimensions.

## 4.1 Basic Cleaning

```
# Import PySpark SQL functions
from pyspark.sql import functions as F

df_tx = (df_raw
        .select(
            F.trim("Transaction_ID").alias("Transaction_ID"),
            F.to_date(F.col("Date"), "M/d/yyyy").alias("Date"),
            F.trim("Customer_Name").alias("Customer_Name"),
            F.trim("Gender").alias("Gender"),
            F.trim("Customer_Type").alias("Customer_Type"),
            F.trim("Product").alias("Product"),
            F.trim("Product_Category").alias("Product_Category"),
            F.col("Unit_Price").cast("double").alias("Unit_Price"),
            F.col("Quantity").cast("int").alias("Quantity"),
            F.col("Discount").cast("double").alias("Discount"),
            F.trim("City").alias("City"),
            F.trim("State").alias("State"),
            F.trim("Country").alias("Country")
        )
    )

display(df_tx.limit(5))
```

### Purpose of this step:

- Trim Whitespace: Removes accidental spaces from strings (e.g., converts " M " to "M") to ensure data consistency.
- Type Casting: Converts numerical columns (Quantity, Price) from text to numbers so we can perform calculations later.
- Date Parsing: Converts the raw date string into a proper Date object for time-based analysis.

## 4.2 Create `Dim_Customer`

We create one row per unique customer and assign a surrogate key based on **first appearance** in the raw dataset.

**Window functions** assign surrogate keys based on a defined ordering (e.g., first appearance in the data). This is a common approach to produce stable, reproducible IDs. The code structure for **4.3, 4.4, and 4.5 follows a similar pattern**: group → aggregate → window order → generate key.

```
from pyspark.sql.window import Window

# 1) Find each customer's first appearance in the raw data
#   groupBy: group rows by customer attributes (one group per
#   unique customer)
#   agg: aggregate function - calculate metrics for each group
#   F.min("Transaction_ID"): find the smallest Transaction_ID in
#   each group
#   .alias("first_tx"): name this calculated column "first_tx"
cust_first = (df_tx
    .groupBy("Customer_Name", "Gender", "Customer_Type")
    .agg(F.min("Transaction_ID").alias("first_tx"))
)

# 2) Define window ordering rule (sort by first appearance)
#   Window.orderBy("first_tx"): arrange customers by their first
#   transaction
w_cust = Window.orderBy("first_tx")

# 3) Assign a numeric surrogate key, then format as C101, C102,
#   ...
#   withColumn: add a new column to the DataFrame
#   dense_rank().over(w_cust): assign sequential numbers
#   (1,2,3...) based on w_cust ordering
#   F.format_string("C%03d", ...): format number as C101, C102,
#   etc. (%03d = 3-digit zero-padded)
dim_customer = (cust_first
    .withColumn("cust_num", F.dense_rank().over(w_cust))
    .withColumn("CustomerID", F.format_string("C%03d",
F.col("cust_num") + 100))
    .select(
        "CustomerID",
        F.col("Customer_Name").alias("CustomerName"),
        F.col("Gender"),
        F.col("Customer_Type").alias("CustomerType")
    )
)

display(dim_customer)
```

|   | <sup>A</sup> <sub>C</sub> CustomerID | <sup>A</sup> <sub>C</sub> CustomerName | <sup>A</sup> <sub>C</sub> Gender | <sup>A</sup> <sub>C</sub> CustomerType |
|---|--------------------------------------|----------------------------------------|----------------------------------|----------------------------------------|
| 1 | C101                                 | John Smith                             | M                                | Regular                                |
| 2 | C102                                 | Emily Johnson                          | F                                | Premium                                |
| 3 | C103                                 | Marie Tremblay                         | F                                | Regular                                |
| 4 | C104                                 | Michael Brown                          | M                                | Premium                                |



 4 rows | 1.61s runtime

### 4.3 Create `Dim_Date`

We create one row per unique date and assign `DateID` starting from 1. We also derive `Month` and `Year`.

```
w_date = Window.orderBy("Date")

dim_date = (df_tx
  .select("Date")
  .dropDuplicates()
  .withColumn("DateID", F.dense_rank().over(w_date))
  .withColumn("Month", F.month("Date"))
  .withColumn("Year", F.year("Date"))
  .select(
    "DateID",
    F.date_format("Date", "M/d/yyyy").alias("Date"),
    "Month",
    "Year"
  )
)

display(dim_date)
```

Table ▾ +

|   | <sup>1</sup> <sub>3</sub> DateID | <sup>A</sup> <sub>C</sub> Date | <sup>1</sup> <sub>3</sub> Month | <sup>1</sup> <sub>3</sub> Year |
|---|----------------------------------|--------------------------------|---------------------------------|--------------------------------|
| 1 | 1                                | 1/15/2025                      | 1                               | 2025                           |
| 2 | 2                                | 1/20/2025                      | 1                               | 2025                           |
| 3 | 3                                | 2/5/2025                       | 2                               | 2025                           |
| 4 | 4                                | 2/10/2025                      | 2                               | 2025                           |



 4 rows | 1.34s runtime

## 4.4 Create `Dim_Location`

We create one row per unique location (City, State, Country) and assign `LocationID` based on **first appearance** in the raw dataset.

```
loc_first = (df_tx
    .groupBy("City", "State", "Country")
    .agg(F.min("Transaction_ID").alias("first_tx"))
)

w_loc = Window.orderBy("first_tx")

dim_location = (loc_first
    .withColumn("LocationID", F.dense_rank().over(w_loc))
    .select("LocationID", "City", "State", "Country")
)

display(dim_location)
```

| Table ▾ + |                                        |                                  |                                   |                                     |  |
|-----------|----------------------------------------|----------------------------------|-----------------------------------|-------------------------------------|--|
|           | 1 <sup>2</sup> <sub>3</sub> LocationID | A <sup>B</sup> <sub>C</sub> City | A <sup>B</sup> <sub>C</sub> State | A <sup>B</sup> <sub>C</sub> Country |  |
| 1         | 1                                      | Seattle                          | WA                                | USA                                 |  |
| 2         | 2                                      | Portland                         | OR                                | USA                                 |  |
| 3         | 3                                      | Vancouver                        | BC                                | Canada                              |  |
| 4         | 4                                      | Chicago                          | IL                                | USA                                 |  |



 4 rows | 1.11s runtime

## 4.5 Create Dim\_Product

We create one row per unique product and assign `ProductID` based on **first appearance** in the raw dataset. (Note: the product dimension stores descriptive attributes; pricing remains in the fact table as measures.)

```
prod_first = (df_tx
    .groupBy("Product", "Product_Category")
    .agg(F.min("Transaction_ID").alias("first_tx"))
)

w_prod = Window.orderBy("first_tx")

dim_product = (prod_first
    .withColumn("ProductID", F.dense_rank().over(w_prod))
    .select(
        "ProductID",
        F.col("Product").alias("ProductName"),
        F.col("Product_Category").alias("Category")
    )
)

display(dim_product)
```

Table ▾ +

|   | $1^2_3$ ProductID | $A^B_C$ ProductName | $A^B_C$ Category |
|---|-------------------|---------------------|------------------|
| 1 | 1                 | Wireless Earbuds    | Electronics      |
| 2 | 2                 | Yoga Mat            | Fitness          |
| 3 | 3                 | Coffee Maker        | Home             |
| 4 | 4                 | Smart Watch         | Electronics      |



 4 rows | 1.14s runtime

## 4.6 Create Fact\_Sales

We join the transaction data to each dimension to obtain foreign keys. Then we compute the measures:

- $\text{TotalPrice} = \text{Unit\_Price} * \text{Quantity}$
- $\text{ActualPrice} = \text{TotalPrice} * (1 - \text{Discount}/100)$  (Discount is a percentage, e.g., 5 means 5%)

```

# Join to customer
tx_keyed = (df_tx
    .join(dim_customer.select("CustomerID",
        F.col("CustomerName").alias("Customer_Name"),
        "Gender",
        F.col("CustomerType").alias("Customer_Type")),
        on=["Customer_Name", "Gender", "Customer_Type"],
        how="left")
    )

# Join to date (convert dim_date.Date back to date type for join)
dim_date_join = dim_date.select("DateID",
    F.to_date(F.col("Date"), "M/d/yyyy").alias("Date"))
tx_keyed = tx_keyed.join(dim_date_join, on=["Date"], how="left")

# Join to location
tx_keyed = tx_keyed.join(dim_location, on=["City", "State",
    "Country"], how="left")

# Join to product
tx_keyed = (tx_keyed
    .join(dim_product.select("ProductID",
        F.col("ProductName").alias("Product"),
        F.col("Category").alias("Product_Category")),
        on=["Product", "Product_Category"], how="left")
    )

# Create SalesID (1..N)
w_sales = Window.orderBy("Transaction_ID")

fact_sales = (tx_keyed
    .withColumn("SalesID", F.dense_rank().over(w_sales))
    .withColumn("TotalPrice", F.round(F.col("Unit_Price") *
    F.col("Quantity"), 2))
    .withColumn("ActualPrice", F.round(F.col("TotalPrice") * (1 -
    F.col("Discount") / 100.0), 2))
    .select(
        "SalesID", "ProductID", "CustomerID", "DateID", "LocationID",
        "Quantity", "TotalPrice", "Discount", "ActualPrice"
    )
    )

display(fact_sales)

```



|    | 1.1 SalesID | 1.1 ProductID | 1.1 CustomerID | 1.1 DateID | 1.1 LocationID | 1.1 Quantity | 1.2 TotalPrice | 1.2 Discount | 1.2 ActualPrice |
|----|-------------|---------------|----------------|------------|----------------|--------------|----------------|--------------|-----------------|
| 1  | 1           | 1             | C101           | 1          | 1              | 1            | 79.99          | 0            | 79.99           |
| 2  | 2           | 2             | C102           | 1          | 2              | 2            | 59.98          | 0            | 59.98           |
| 3  | 3           | 3             | C103           | 2          | 3              | 1            | 99.99          | 10           | 89.99           |
| 4  | 4           | 4             | C104           | 2          | 4              | 1            | 249.99         | 0            | 249.99          |
| 5  | 5           | 1             | C102           | 3          | 2              | 1            | 79.99          | 5            | 75.99           |
| 6  | 6           | 2             | C101           | 3          | 1              | 3            | 89.97          | 0            | 89.97           |
| 7  | 7           | 3             | C104           | 4          | 4              | 1            | 99.99          | 0            | 99.99           |
| 8  | 8           | 4             | C103           | 4          | 3              | 1            | 249.99         | 25           | 187.49          |
| 9  | 9           | 3             | C101           | 1          | 1              | 2            | 199.98         | 15           | 169.98          |
| 10 | 10          | 1             | C102           | 2          | 2              | 1            | 79.99          | 0            | 79.99           |
| 11 | 11          | 2             | C103           | 3          | 3              | 4            | 119.96         | 10           | 107.96          |
| 12 | 12          | 1             | C104           | 4          | 4              | 2            | 159.98         | 0            | 159.98          |
| 13 | 13          | 4             | C102           | 1          | 2              | 1            | 249.99         | 20           | 199.99          |
| 14 | 14          | 2             | C101           | 2          | 1              | 2            | 59.98          | 0            | 59.98           |
| 15 | 15          | 3             | C104           | 3          | 4              | 3            | 299.97         | 25           | 224.98          |

20 rows | 3.64s runtime

## Step 5: Load (Save as Delta Tables)

We store the final tables as **Delta tables**.

Delta tables are a storage format that provides:

- **ACID transactions:** Multiple users can read/write simultaneously without conflicts.
- **Data versioning:** Track changes and roll back to previous versions if needed.
- **Performance:** Optimized for fast queries on large datasets.

Delta is the default and recommended table format in Databricks for reliable data warehousing.

```

# 1) Get current username and format it for database naming
#     regexp_replace replaces non-alphanumeric characters with
#     underscores
#     .first()[0] extracts the first row, first column value
username = spark.sql("SELECT regexp_replace(current_user(), '^[a-
zA-Z0-9]', '_')").first()[0]

# 2) Create a user-specific database to avoid table name
#     conflicts
spark.sql(f"CREATE DATABASE IF NOT EXISTS db_{username}")

# 3) Switch to this database (all subsequent tables will be saved
#     here)
spark.sql(f"USE db_{username}")

# 4) Save each DataFrame as a Delta table
#     .format("delta") = use Delta table format
#     .mode("overwrite") = replace table if it exists (safe for
#     re-runs)
#     .saveAsTable() = register as a queryable table
dim_customer.write.format("delta").mode("overwrite").saveAsTable(
    "Dim_Customer")
dim_date.write.format("delta").mode("overwrite").saveAsTable("Dim
_Date")
dim_location.write.format("delta").mode("overwrite").saveAsTable(
    "Dim_Location")
dim_product.write.format("delta").mode("overwrite").saveAsTable("
Dim_Product")
fact_sales.write.format("delta").mode("overwrite").saveAsTable("F
act_Sales")

print(f"Tables saved in database: db_{username}")

```

## Step 6: Validate (Basic Checks)

### 6.1 Row count checks

Verify that all tables were created successfully by checking the number of rows in each table. This ensures the ETL process completed without data loss.

```
print("Dim_Customer:", spark.table("Dim_Customer").count())
print("Dim_Date      :", spark.table("Dim_Date").count())
print("Dim_Location:", spark.table("Dim_Location").count())
print("Dim_Product  :", spark.table("Dim_Product").count())
print("Fact_Sales   :", spark.table("Fact_Sales").count())
```

```
Dim_Customer: 4
Dim_Date      : 4
Dim_Location: 4
Dim_Product  : 4
Fact_Sales   : 20
```

## 6.2 Star schema join validation (SQL)

Test that the fact table can join with dimension tables correctly. This query aggregates sales revenue by product category, demonstrating that the star schema relationships work as expected.

```
%sql
SELECT
  p.Category,
  SUM(f.ActualPrice) AS Total_Revenue,
  SUM(f.Quantity)   AS Total_Quantity
FROM Fact_Sales f
JOIN Dim_Product p
  ON f.ProductID = p.ProductID
GROUP BY p.Category
ORDER BY Total_Revenue DESC;
```

| Table ▾ |                                    | +                                       |                                          |
|---------|------------------------------------|-----------------------------------------|------------------------------------------|
|         | <sup>A</sup> <sub>C</sub> Category | <sup>1</sup> <sub>2</sub> Total_Revenue | <sup>1</sup> <sub>3</sub> Total_Quantity |
| 1       | Electronics                        | 1363.4                                  | 10                                       |
| 2       | Home                               | 684.93                                  | 8                                        |
| 3       | Fitness                            | 529.31                                  | 19                                       |



 3 rows | 2.29s runtime

[Previous](#)

[Week 2 - Software Setup](#)

[Next](#)

[Task Demonstration Video \(Python\)](#)

Last updated 1 month ago